

8h às 10h

Assista aos vídeos e leia os artigos indicados antes do encontro.

Autoestudo: uma trilha, não uma lista

1

Mais exemplos de modelagem, rebalanceamento e o Imputer do sklearn (vídeo)

3

Formas de lidar com o desbalanceamento de classes (Machine Learning Mastery)

2

A maldição da dimensionalidade e o PCA em Python (Alura)

4

Curva ROC e AUC: como avaliar um modelo (vídeo)

| Stand-up daily: em pé, direto ao ponto

15 min

10:00 → 10:15

1

Em pé

A daily é rápida — todos de pé.

2

Tasks do projeto

O que cada um fez e o que falta.

3

Cronograma do dia

Definir prioridades e metas de hoje.

Objetivo: alinhar o time e destravar o dia.

DIAGRAMA DO MÓDULO



AULA 9 • SEMANA 7

Problemas Comuns de Modelagem e Feature Engineering

Rebalanceamento • Imputer • Maldição da Dimensionalidade • PCA • ROC/AUC

Prof^a Dra. Crishna Irion

Agenda do Encontro

1

Conceito

5 blocos: da modelagem ao Imputer, PCA e ROC/AUC

2

Aplicação prática

Overfitting, underfitting e domain knowledge
Desbalanceamento: oversampling e undersampling
Dados faltantes e o Imputer do sklearn
Maldição da dimensionalidade e PCA
Avaliação de modelos: curva ROC e AUC
Boas práticas de generalização do modelo

3

Código

Rebalanceamento com imblearn
(Over/UnderSampling)
Tratamento de nulos com SimpleImputer
Redução de dimensionalidade com PCA
Curva ROC, AUC e roc_auc_score

PARTE 1

Problemas Comuns de Modelagem

Reconhecer quando um modelo aprendeu demais (ou de menos) — e por que isso importa.

Por que um modelo "perfeito" falha no mundo real?

```
Treino      -> acuracia: 99%  
Teste       -> acuracia: 61%  
Mesmo modelo. Dados diferentes.  
Aprovamos esse modelo?
```

O modelo só decorou uma amostra.

Por que a acurácia caiu tanto no teste?
O modelo aprendeu um padrão ou só "decorou" exemplos?
Como saber se ele vai generalizar amanhã?

Em duplas: o que vocês fariam antes de colocar esse modelo em produção?

Três formas de um modelo se ajustar aos dados

1

Underfitting

modelo simples demais;
não captura nem o
padrão do treino

2

Ajuste ideal

captura o padrão real;
generaliza bem para
dados novos

3

Overfitting

"decorou" o treino; erra
feio em dados novos

A meta não é acertar 100% do treino — é generalizar para o que vem depois.

Um modelo pode ir bem no treino e mal no mundo real — por quê?

NO TREINO (dados conhecidos)

- o modelo já viu esses exemplos dezenas de vezes
- métricas ficam artificialmente altas
- fácil confundir "decorou" com "aprendeu"
- domain knowledge fraco: ninguém questiona o resultado

NO MUNDO REAL (dados novos)

- situações e combinações nunca vistas no treino
- métricas caem se o modelo só memorizou
- aqui é onde o modelo é realmente testado
- domain knowledge forte: sabemos o que esperar e o que desconfiar

Domain knowledge: entender o problema real evita um modelo "bom só no laboratório".

PARTE 2

Desbalanceamento de Classes

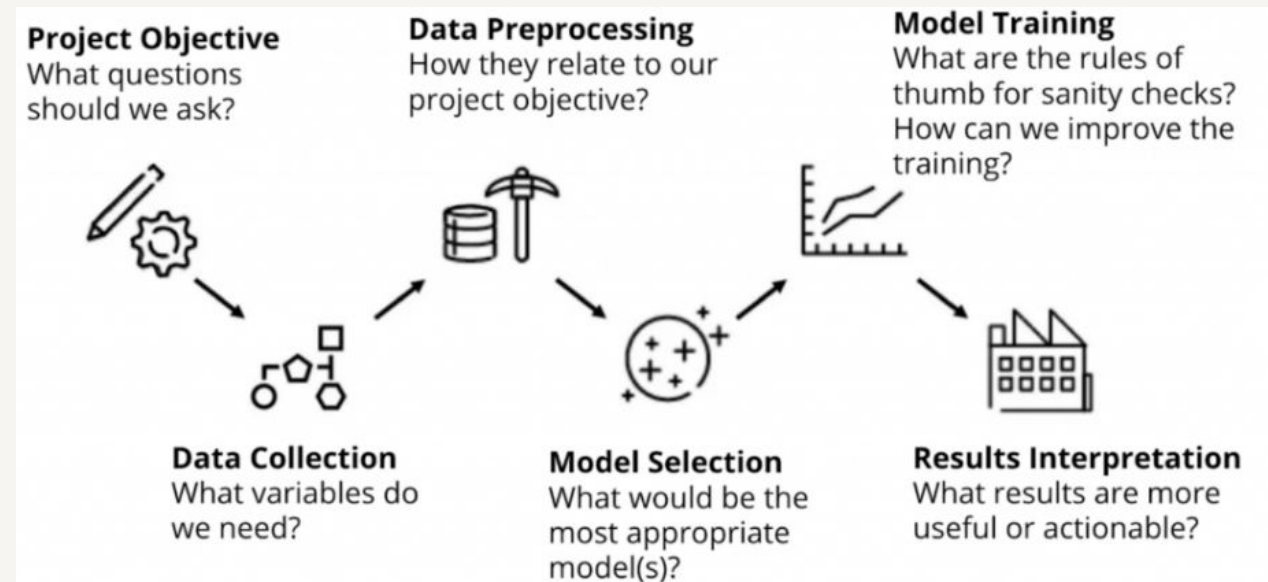
Quando uma classe domina os dados, o modelo aprende a ignorar a minoria.

Conhecimento da classe de domínio

Para que serve?

Orienta decisões em toda a etapa do pipeline: formulação, coleta, pré-processamento e modelagem.

Como ajuda o projeto?



Dados Desbalanceados

Uma classe (minoritária) aparece bem menos que as outras (majoritárias) no dataset.

Paradoxo da Acurácia ocorre quando os parâmetros do algoritmo não diferenciam a classe minoritária das demais categorias, acreditando que estão agregando resultado devido à aparente alta acurácia.

Maior

~87% dos vinhos são 'Ruim' no exemplo da Turing Talks

Menor

~13% são 'Bom' — e é essa classe que mais importa

Desbalanceamento: quando uma classe domina o dataset

o problema

poucos exemplos da classe
que mais importa

por que atrapalha

- 1 · acurácia vira uma métrica enganosa
- 2 · o modelo aprende a ignorar a minoria

Por que acontece

Fraude, doença rara, churn: a classe "interessante" é sempre a minoria natural dos dados. Um modelo "preguiçoso" que sempre prevê a classe majoritária já acerta 95%+ de acurácia — e é inútil.

O recall da classe minoritária é quem conta a verdade sobre o modelo.

Ponto de atenção: sempre olhe a matriz de confusão, não só a acurácia.

Duas formas de rebalancear: Oversampling vs Undersampling

OVERSAMPLING

"crie mais exemplos da classe rara"

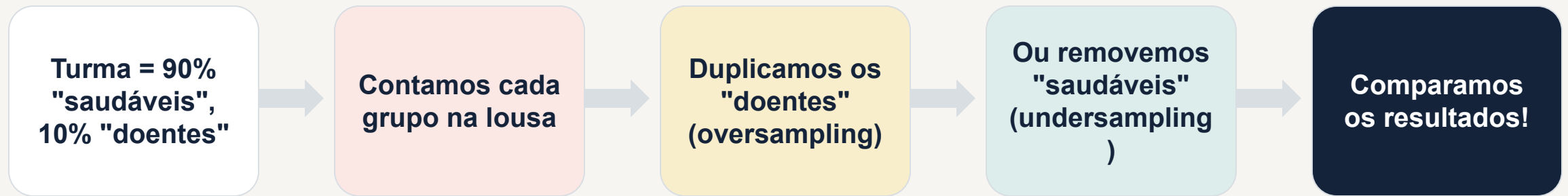
duplica ou gera exemplos sintéticos (ex.: SMOTE)
mantém todo o dado da classe majoritária
risco: overfitting em exemplos repetidos

UNDERSAMPLING

"reduza a classe majoritária"

remove exemplos da classe dominante
dataset final menor e mais rápido de treinar
risco: perder informação relevante

Dinâmica: a turma rebalanceia um dataset desbalanceado



Sem código: a turma sente na pele por que balancear os dados muda o resultado.

Código: rebalanceando com IMBLEARN



1. Diagnosticar o desbalanceamento

Sempre comece contando quantos exemplos há de cada classe.

```
print(y.value_counts())  
# 0    9400    (maioria)  
# 1     600    (minoria)  
# proporcao 94% x 6%: desbalanceado
```



2. Rebalancear com imblearn

RandomOverSampler duplica; RandomUnderSampler remove.

```
from imblearn.over_sampling import RandomOverSampler  
ros = RandomOverSampler(random_state=42)  
X_res, y_res = ros.fit_resample(X, y)  
print(y_res.value_counts())  
# 0    9400 | 1    9400  -> balanceado
```

Oversampling e undersampling ajustam a proporção de classes antes do treino.

PARTE 3

Dados Faltantes: o Imputer

Tratar valores nulos sem descartar informação valiosa.



| Estratégias de imputação: qual valor usar no lugar do nulo?

NaN → ?

SimpleImputer substitui valores nulos por uma estimativa razoável.

- 1 Média (mean)**
ideal para variáveis numéricas sem outliers fortes
- 2 Mediana (median)**
mais robusta a outliers do que a média
- 3 Mais frequente / constante**
para variáveis categóricas, ou um valor fixo (ex.: 0)

Código: tratando nulos com SimpleImputer

1. Detectar valores nulos

Sempre visualize quantos nulos existem antes de tratar.

```
print(df.isna().sum())  
# idade          120 nulos
```

2. Imputar com SimpleImputer

Ajuste (fit) no treino e aplique (transform) o mesmo imputer no teste.

```
from sklearn.impute import SimpleImputer  
num_imp = SimpleImputer(strategy='median')  
cat_imp = SimpleImputer(strategy='most_frequent')  
  
df[num_cols] = num_imp.fit_transform(df[num_cols])  
df[cat_cols] = cat_imp.fit_transform(df[cat_cols])  
  
# no teste, use so .transform():  
# df_teste[num_cols] = num_imp.transform(...)  
  
print(df.isna().sum().sum())  
# 0 -> nenhum nulo restante
```

PARTE 4

Maldição da Dimensionalidade e PCA

Quando colunas demais atrapalham: como o PCA resume os dados sem perder o essencial.

O que acontece quando os dados têm colunas demais

1

Esparsidade

com muitas colunas,
os dados ficam espalhados e
vazios

2

Distâncias perdem sentido

em alta dimensão,
tudo fica "igualmente longe"

3

Mais risco de overfitting

muitas variáveis, poucos
exemplos:
o modelo decora em vez de
aprender

Mais colunas nem sempre é melhor — às vezes é preciso resumir a informação.

PCA: resumir muitas colunas em poucas, sem perder o essencial

10D → 2D

- PCA cria novas variáveis (componentes principais)
- cada componente é uma combinação das variáveis originais
- os primeiros componentes concentram a maior parte da variância
- menos colunas, quase a mesma informação

PCA não escolhe colunas — cria novas, ordenadas pela variância que explicam.

Código: reduzindo dimensões com PCA



1. Padronizar os dados

PCA é sensível à escala — sempre padronize antes.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```



2. Aplicar o PCA

Escolha quantos componentes manter pela variância explicada.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print(pca.explained_variance_ratio_)

plt.scatter(X_pca[:,0], X_pca[:,1])
plt.show()

# [0.62, 0.24] -> 86% da variancia
```

PCA exige dados padronizados: escalas diferentes distorcem a variância.

PARTE 5

Avaliação de Modelos: Curva ROC e AUC

Ir além da acurácia: descobrir se o modelo realmente separa bem as classes.

A curva ROC nasce de duas taxas em tensão

TAXA DE VERDADEIROS POSITIVOS

(sensibilidade / recall) dos casos positivos,
quantos o modelo acertou
sobe quando o modelo é mais "agressivo" ao
prever positivo
eixo Y da curva ROC
queremos essa taxa alta

TAXA DE FALSOS POSITIVOS

dos casos negativos, quantos o modelo errou
como positivos
sobe junto com a taxa de verdadeiros positivos
eixo X da curva ROC
queremos essa taxa baixa

**AUC resume a curva inteira: 0,5 é aleatório, 1,0 é
separação perfeita.**

Código: curva ROC e AUC



1. Gerar as probabilidades

Use `predict_proba`, não `predict` — a curva precisa de um score contínuo.

```
from sklearn.metrics import roc_curve, roc_auc_score
probas = modelo.predict_proba(X_teste)
y_score = probas[:, 1]
# probabilidade da classe positiva
# nunca use predict() aqui
fpr, tpr, limiares = roc_curve(y_teste, y_score)
```



2. Calcular a curva e o AUC

`roc_curve` devolve os pontos; `roc_auc_score` resume tudo em um número.

```
auc = roc_auc_score(y_teste, y_score)
print(f"AUC = {auc:.2f}")

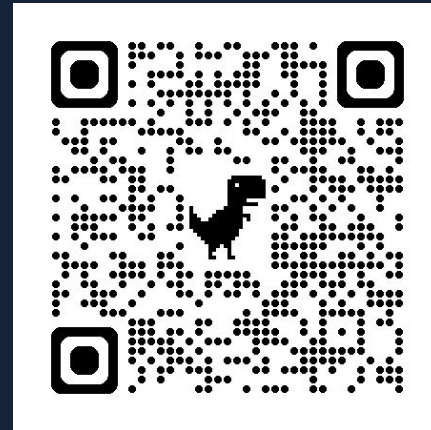
plt.plot(fpr, tpr, label=f"AUC={auc:.2f}")
plt.plot([0,1], [0,1], "--") # linha do acaso
```

`roc_auc_score` resume a curva inteira em um único número, de 0,5 a 1,0.

PARTE 6

Mão na Massa

Vamos codar?

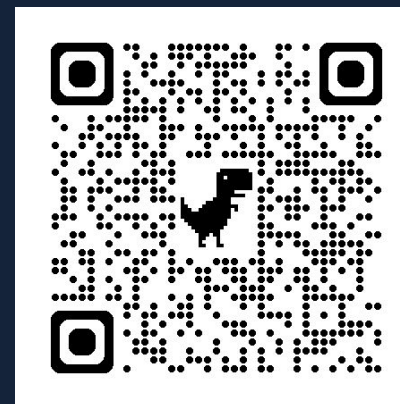


<https://encurtador.com.br/lCfp>

PARTE 6

Mão na Massa

O gabarito...



<https://encurtador.com.br/JaAP>

Dúvidas? Vamos conversar.

Profª Dra. Crishna Irion

crishna.irion@prof.inteli.edu.br

